1 Introduction to Automata and Computability

Automata and Computability is a core area of theoretical computer science that studies abstract models of computation, the limits of what can be computed, and the classification of problems based on their solvability and resource requirements. Automata theory deals with mathematical models of machines (automata) that process inputs according to rules, while computability explores what problems are solvable by algorithms and which are not.

1.1 Key Concepts

- **Automaton**: A theoretical machine that accepts or rejects inputs based on states and transitions.
- **Languages**: Sets of strings over an alphabet (e.g., binary strings $\{0,1\}^*$).
- **Hierarchy**: Chomsky Hierarchy classifies languages by complexity (regular, context-free, context-sensitive, recursively enumerable).
- **Computability**: Determines if a problem has an algorithmic solution.
- **Importance**: Foundations for compilers, AI, formal verification, cryptography, and understanding computational limits.

1.2 History

- **1930s–1940s**: Alan Turing's Turing Machine (1936) formalized computation. Alonzo Church's lambda calculus and Emil Post's models showed equivalence.
- **1950s**: Noam Chomsky's hierarchy (1956). Stephen Kleene's regular expressions.
- **1960s–1970s**: Undecidability proofs (e.g., Rice's Theorem). Complexity theory emerges.
- **As of 2025**: Applications in quantum automata, DNA computing, and AI safety (e.g., verifying undecidable properties in large models).

This field proves some problems are inherently unsolvable, like the Halting Problem, influencing software design and AI ethics.

2 Finite Automata

Finite Automata (FA) are the simplest models, with finite states and no memory beyond the current state.

2.1 Deterministic Finite Automaton (DFA)

- **Definition**: $M = (Q, \Sigma, \delta, q_0, F)$
  - Q: Finite states.
  - Σ: Alphabet (input symbols).
  - δ: Transition function ($\delta : Q \times \Sigma \to Q$).
  - q_0: Start state.
  - F: Accept states.
- **Behavior**: Reads input left-to-right, changes states via δ. Accepts if ends in F.

- **Example**: DFA for strings with even number of 0s over $\{0, 1\}$.
  - States: q_{even} (start/accept), q_{odd}.
  - Transitions: On 0, toggle; on 1, stay.
- **Properties**: Deterministic (one path per input). Recognizes regular languages.

2.2 Nondeterministic Finite Automaton (NFA)

- **Difference**: $\delta : Q \times \Sigma \to 2^Q$ (multiple next states).
- **Epsilon-NFA**: Allows ε-transitions (no input consumed).
- **Theorem**: DFA $\equiv$ NFA $\equiv$ ε-NFA (power-set construction converts NFA to DFA, exponential blowup possible).
- **Advantages**: NFAs are more concise for design.

2.3 Minimization

- **Myhill-Nerode Theorem**: Equivalence classes for indistinguishable states.
- **Algorithm**: Hopcroft's ($O(n \log n)$) or table-filling method to merge equivalent states.

3 Regular Languages and Expressions

Regular languages are those recognized by finite automata.

3.1 Properties

- **Closure**: Closed under union, concatenation, Kleene star ($*$), complement, intersection.
- **Pumping Lemma**: For regular L, strings longer than pumping length p can be pumped (xyz with $|xy| \leq p$, $|y| \geq 1$, $xy^k z \in L$ for $k \geq 0$). Used to prove non-regularity (e.g., $\{0^n 1^n\}$ is not regular).

3.2 Regular Expressions (Regex)

- **Definition**: Algebraic way to describe regular languages.
  - Base: ε (empty), a (symbol), $\emptyset$ (empty set).
  - Operations: Union ($|$), concatenation ($\cdot$), star ($*$).
- **Example**: $(0|1)^*0(0|1)^*$ for strings with at least one 0.
- **Equivalence**: Regex $\equiv$ FA (Kleene's Theorem: convert via NFAs).

3.3 Applications

- Pattern matching (grep, Python `re` module), lexical analysis in compilers, network protocols.

4 Pushdown Automata and Context-Free Languages

For languages with nesting (e.g., balanced parentheses).

4.1 Pushdown Automaton (PDA)

- **Definition**: FA + stack (infinite, LIFO).
  - $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$
  - Γ: Stack alphabet, Z_0: Initial stack symbol.
  - $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \to 2^{Q \times \Gamma^*}$ (nondeterministic).
- **Behavior**: Reads input, pushes/pops stack symbols.
- **Deterministic PDA (DPDA)**: Subset, less powerful (e.g., can't recognize palindromes nondeterministically).

4.2 Context-Free Grammars (CFG)

- **Definition**: $G = (V, \Sigma, R, S)$
  - V: Variables, Σ: Terminals, R: Rules ($A \to w$ where $A \in V$, $w \in (V \cup \Sigma)^*$), S: Start.
- **Example**: $S \to (S) \mid \varepsilon$ for balanced parentheses.
- **Parse Trees**: Represent derivations.
- **Equivalence**: CFG $\equiv$ PDA (convert grammar to PDA via stack simulations).
- **Chomsky Normal Form (CNF)**: Rules $A \to BC$ or $A \to a$ (useful for parsing).

4.3 Properties

- **Closure**: Union, concatenation, star (not complement or intersection).
- **Pumping Lemma for CFLs**: $uvxyz$ with $|vy| \geq 1$, $|vxy| \leq p$, $uv^kxy^kz \in L$.
- **Non-CFL Proof**: $\{a^nb^nc^n\}$ not CFL (pumping breaks equality).

4.4 Parsing

- **Top-Down**: LL(k) parsers (predictive).
- **Bottom-Up**: LR(k) (shift-reduce, used in Yacc/Bison).
- **Ambiguity**: Grammars with multiple parse trees; resolve via precedence.

Applications: Programming languages (syntax), XML/HTML parsing, natural language processing.

5 Turing Machines

The most powerful model, basis for computability.

5.1 Deterministic Turing Machine (TM)

- **Definition**: $M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$
  - Γ: Tape alphabet (includes Σ and blank B).
  - $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ (state, read $\to$ new state, write, move).
  - Tape: Infinite, two-way.
- **Behavior**: Starts with input on tape, halts if enters accept/reject state.
- **Example**: TM for $\{ww \mid w \in \{0,1\}^*\}$ (palindromes): Copy, compare.

5.2 Variants

- **Nondeterministic TM (NTM)**: Multiple transitions; accepts if any path accepts.
- **Multi-Tape TM**: Equivalent to single-tape (simulate with markers).
- **Theorem**: All variants equivalent in power (Church-Turing Thesis: Any effective computation can be done by a TM).

5.3 Extensions

- **Universal TM**: Simulates any TM (like a general-purpose computer).
- **Linear Bounded Automaton (LBA)**: Tape limited to input size; recognizes context-sensitive languages.

6 Computability Theory

Studies what can be computed.

6.1 Recursive Languages

- **Recursively Enumerable (RE)**: Accepted by TM (may loop forever on non-members).
- **Recursive (Decidable)**: TM halts with yes/no on all inputs.
- **Co-RE**: Complement of RE.

6.2 Decidability

- **Decidable Problems**: A_{TM} (acceptance by specific TM)? No, undecidable.
- **Examples**:
  - DFA equivalence: Decidable (minimize and compare).
  - CFG emptiness: Decidable.
  - Halting Problem: Undecidable (diagonalization: Assume H solves halt; construct D that halts iff H says it doesn't—contradiction).

6.3 Reductions

- **Mapping Reduction**: Problem $A \leq_m B$ if solver for B solves A.
- **Turing Reduction**: $A \leq_T B$ if oracle for B solves A.
- Used to prove undecidability (e.g., Post Correspondence Problem reduces to Halting).

6.4 Rice's Theorem

- Any non-trivial property of RE languages is undecidable (e.g., "Does L(TM) contain 'hello'?").

6.5 Oracle Machines

- TMs with black-box solvers for undecidable problems; leads to degrees of unsolvability.

7 Undecidability and Incompleteness

- **Undecidable Problems**: Busy Beaver (max steps before halt), Kolmogorov complexity (shortest program for string).
- **Gödel's Incompleteness Theorems**: Formal systems can't prove all truths (1931); links to computability.
- **Real-World Implications**: Can't automatically verify arbitrary programs (e.g., no perfect bug finder).

8 Complexity Theory (Intersection with Computability)

While separate, often studied together.

- **Time/Space Complexity**: For TMs.
- **P**: Deterministic polynomial time.
- **NP**: Nondeterministic poly time (verifiable in poly time).
- **P vs. NP**: Open problem (as of 2025); if P=NP, cryptography breaks.
- **NP-Complete**: Hardest in NP (e.g., SAT, via Cook-Levin Theorem).
- **Beyond**: EXP, PSPACE, undecidable.

9 Advanced Topics

- **Quantum Automata**: Quantum Finite Automata (QFA); more powerful for some languages.
- **Cellular Automata**: Grid-based (e.g., Conway's Game of Life); models computation in parallel.
- **Lambda Calculus**: Functional model, equivalent to TMs.
- **DNA/Membrane Computing**: Biological models for massive parallelism.
- **Hypercomputation**: Theoretical models beyond TMs (e.g., oracle machines), but not physically realizable.
- **As of 2025**: Automata in ML (e.g., neural TMs), computability in quantum error correction.

10 Applications

- **Compilers**: Lexers (FA/regex), parsers (PDA/CFG).
- **Verification**: Model checking with automata for software/hardware.
- **AI**: State machines in games/robots; undecidability in AGI safety.
- **Cryptography**: One-way functions from complexity.
- **Biology**: Automata model gene regulation.
- **Networks**: Protocol verification with TMs.

11 Best Practices and Learning Tips

- **Proofs**: Master induction, contradiction for theorems.
- **Simulation**: Use tools to build automata (e.g., JFLAP).
- **Common Pitfalls**: Forgetting ε-closures in NFAs; assuming decidability.
- **Practice**: Prove languages regular/non-regular; design TMs for problems.